4
Real-Time Message Translation

Now we venture into looking at real-world use cases and actual problems that can be implemented and solved with Spring AI using various AI models. Being familiar and comfortable with Spring AI's APIs presented in *Chapters 2* and *3* will help you get the most out of the next few chapters, starting in this chapter.

In this chapter, we present real-time language translation applied to messaging, such as chat. In *Chapter 5*, we demonstrate how to beat, err... "implement" Shazam. And finally, in *Chapter 6*, we demonstrate the behavior of two different AI models competing against each other in the classic game of Connect4.

Communication has always been a particularly fascinating topic, how languages evolved, and humans learned to converse verbally as well as through body language, gestures and even touch. We communicate through a plethora of mediums, dating back to carrier pigeons, morse code, telegram and sending post in the mail. Then came telephones, email, chat and video conference calling. I imagine in the not-too-distant future, technology aided telepathy will be a real thing, especially if Elon Musk has anything to say about it. Imagine the little implant inside our heads going off every time we receive another message, "bzzzzt!" Gah!

Recent advances in reasoning models, such as OpenAI's real-time 4o[-mini] Transcribe and Whisper models, along with GPT-5, among others, will be particularly interesting to evaluate as the technology continues to improve in language understanding. After all, these are large "language" models, or LLMS. I say we put them to the test. In this chapter, we discuss the following applications of AI for our use case:

- Real-Time Language Translation for Messaging
- Code Generation
- Language Translation Using AI
- Audio Transcription (Speech-to-Text)
- Text-to-Speech

In this chapter you will learn how to use Spring AI's APIs that we covered in *Chapters 2* and *3* to utilize AI models for language translation, audio transcription, and text-to-speech. These features will be applied to messaging via a chat application. This is a full-featured Spring AI, Spring Boot application. The application requirements are detailed below.

Technical requirements

The real-time language translation messaging app showcased in this chapter is based on Java 17 and Spring Boot 3.5.x. The app is using Spring AI 1.0.x.

You must manually install JDK 17 or later. The Maven POM declares and resolves Spring AI, Spring Boot and all other dependencies required by the app at runtime from Maven Central.

This app uses Ollama for language translation. Specifically, the app used Meta's Llama3.2 model. You can download and install Ollama from https://ollama.com/download.

After you download and install Ollama, you can install Meta's Llama3.2 model, or any other Ollama supported model by name using the following command:

```
$ ollama run llama3.2
```

Use the following command to install an Ollama model without running it:

```
$ ollama pull deepseek-r1:latest
```

It is generally a good idea to try and run the AI model before attempting to access the AI model from Spring AI to make certain the AI model will actually work on your system. Follow the link, https://ollama.com/search, to find more information about Meta's Llama3.2 model as well as other AI models that run on Ollama.

Additionally, the app uses OpenAI's `TTS` model to integrate Text-to-Speech into the app. You must set up an OpenAI account at https://platform.openai.com/signup and purchase credits to make OpenAI API calls.

Create an OpenAI API key and set the following Spring AI property in Spring Boot `application.properties` like so:

```
spring.ai.openai.api-key=<your-openai-api-key>
```

Make sure to be conscious of your use and spend when performing either Audio Transcription or Text-to-Speech operations using the app. The app does not try to do things in the most efficient manner possible, like caching. It simply illustrates the AI use case. Refer to OpenAI's pricing at https://platform.openai.com/docs/pricing for more details.

The GitHub repository module README (https://github.com/PacktPublishing/Building-Intelligent-Applications-With-Spring-AI/tree/main/realtime-chat-translation-app) will provide more instructions and guidance when using the app.



Finally, this example makes use of custom Spring AI extensions, which are covered in more detail in *Chapter 7*. While use of specific extensions is described in the context of the use case, please refer to *Chapter 7* for a complete description of available extensions and their functionality.

Real-Time language translation for messaging

Real-Time Message Translation (**RTMT**) is an extension of instant messaging, such as chat or TXT. The function automates contextually-aware message transformations, like language translation, as a conversation naturally materializes.

We constantly interface and interact with other people, and more recently, ever increasingly, with chatbots. However, we don't always speak the same language. Even if non-native speakers speak our language we are often divided by cultural barriers.

The problem is further complicated by the fact that some languages don't have the same vocabulary. For example there is no direct translation for the English word "awkward" in German. Rather, German uses more precise words to describe the situation, such as "peinlich" meaning embarrassing, or "unangenehm" meaning uncomfortable, where "awkward" covers both meanings simultaneously. "Awwkwaard".

Chatbots introduce further challenges, often times to our own detriment. Given the non-existence of *Artificial General Intelligence* (AGI), AI lacks the ability to "read the situation" using multiple forms of communication, such as body language. With incomplete context and a whole cohort of other problems, Chatbots generally fail to provide any relevant help. As result, and all too frequently, we are left feeling frustrated and our problem still persists.

Although, not all is lost. AI can function as an assistive technology to bridge a few gaps in communication, like language. Human language Interpreters (spoken) and Translators (written) have been doing this for many years. Accessibility features in assistive technologies, such as screen readers, magnification, hearing devices, (live) captions, Apple's VoiceOver, are not new either.

While all the possible use cases of this application are too numerous to list, a common use case is helping customer service representatives when speaking with customers online. The app could also be used in less formal settings, such as for asynchronous message systems like voice mailboxes.

My personal favorite would be for the app to help first responders (e.g. 911 dispatchers) during emergency situations communicate with any caller. The possibilities are endless.

In any case, the main objective and key result for any application of r*eal-time message translation* would be to enable users from diverse backgrounds, speaking different languages, to communicate in a manner that feels natural, as though the user is speaking and listening to someone in their native tongue.

Now that we have defined a desired outcome, let's define a minimal set of requirements for our application.

Application requirements

The primary application requirement is to enable any number of people from diverse backgrounds, speaking many different languages, to converse naturally without having to know how to speak the other participants' language.

In addition, the application must also:

- Require users to enter their name and select a language preference before joining a chat
- A user's language preference will default to the system language setting when no selection is provided
- Users will receive chat messages from other users in their preferred language
- The app assumes users will send chat messages in their preferred language
- Enable users to verbally speak their chat message using their system mic
- Enable chat messages to be audibly read in the user's preferred language

A few additional features and functionality intrinsic to chat applications are also included:

- List users participating in a chat including a presence or active state indicator
- Invite users to participate in a chat by sending the chat session link in email

Code generation

Early in my career, I was a full-stack software engineer writing Java on backend and HTML, JavaScript and CSS presented with **Java Server Pages** (**JSP**) on the frontend. Being a full-stack engineer was certainly much more common in the late 90s and early to mid-2000s than it is today. In my experience, this practice lasted until 2009 when the responsibilities of these roles were divided. From this moment onward, the role of a UI/UX engineer was born.

While I very much appreciate the discipline of **user interface** (**UI**) and **user experience** (**UX**) design, I must admit, I am no expert. Good UI/UX is a very detailed art. It takes years of experience to master in order to design and develop natural, elegant and intuitive interfaces that amaze users with ease. Of course, good UI/UX doesn't precisely fit my statement at the beginning of the book when I said the best kind of technology is never seen, only felt. The UI is very much "seen" and its UX is definitely "felt".Any non-intuitive UI that does not function properly will definitely provoke a response that is often expressed with 3-letter acronyms. Originally, I planned to build the app's user interface from scratch, until I thought, well, I could use AI to generate one.

This book is about AI after all. With the assistance of AI, generating intuitive and interactive user interface (UI) components is relatively quick and easy, if you know what to ask. Also, it would certainly take me much longer to get a decent UI working quickly than if I were to build on a generated one to begin with. My JavaScript and CSS kung fu is by no means legendary.

It usually involves, "how the bleep does that work again", a lot of cursing, and a pound of wax to polish it into something functional, maintainable and useable. But, make no mistake about it, AI is like using the express lane until you hit road construction. You will be slowed down by "prompt hell" having to pave your way forward. The road is the way, but not the destination. In other words, AI is not going to replace good UI/UX design anytime soon. Experienced designers are essential because design requires the human element of creativity and intuition that cannot be taught. It can only be learned through experience and practice. That is why it is an art.

So, to get me started with the UI, I used the following prompt:

"*Generate a nice looking web-based chat UI with soft, translucent, colorful message bubbles between 1 or more users in HTML and JavaScript*"

Yep, that was all it took. It was enough to get me started. OpenAI ChatGPT generated a decent looking UI for a chat app that was functional out-of-the-box:

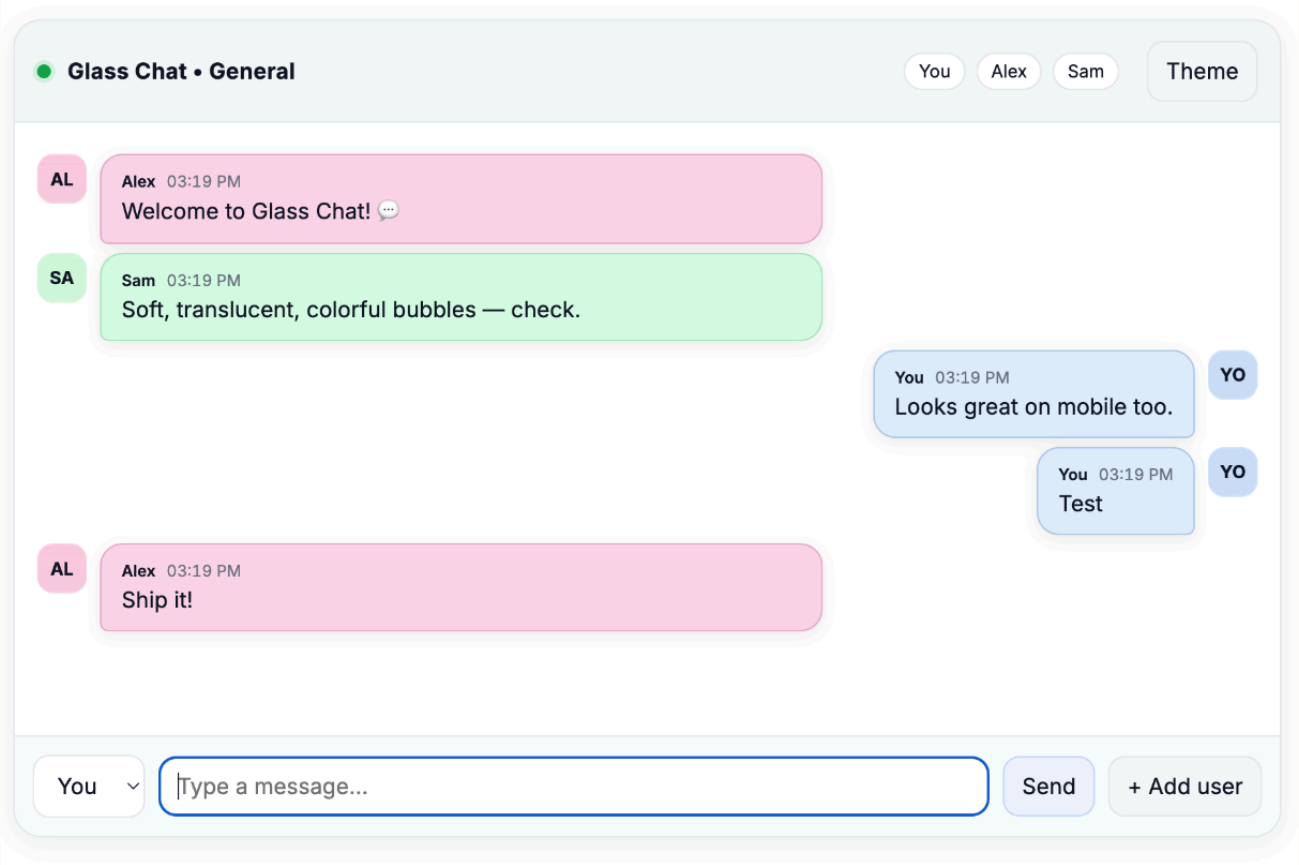


Figure 4.1: OpenAI Generated Chat App

My first AI generated chat UI used a dark theme. That was not going to work if I wanted to include image of the generated result in this book, so I created another prompt to re-generate the UI on a white background:

"*Convert the styles in the CSS to use light colors on a white background*"

OpenAI's ChatGPT was happy to oblige, which was a good thing too, because I asked AI many questions when it came to working out the CSS and JavaScript for the chat interface.

All the prompts I crafted over many attempts of refining the UI are too numerous to list and discuss here, but just know that it was an iterative process.

Using many focused prompts on specific, isolated UI elements and functions works better than trying to get everything right using a single prompt. The key to better outcomes is to experiment and try different things, even using different AI models. I often compared and contrasted OpenAI ChatGPT with Google Gemini, along with a few other models.

You will find a few of the prompts I used in the `src/prompts` directory of the GitHub repository. The original, generated HTML, CSS and JavaScript source are in the `src/main/resources/static/Chat.html.original*` files . You can compare the original files to the final product in `src/main/resources/templates` and `src/main/resources/static` containing the CSS and JavaScript.

I will only discuss bits and pieces of the HTML and JavaScript as it pertains to the functions indirectly tied to the AI model interactions coded with Spring AI on backend, but I will not discuss all of it. Please refer to online resources on HTML, CSS and JavaScript for more guidance. The best source in my opinion is **Mozilla's Developer Network** (**MDN**) at https://developer.mozilla.org/en-US/. After all, they literally wrote the book for the Web.

Language translation using AI

Natural Language Processing (NLP) using computers to process, understand and generate human language is not a new concept. It has been a study and discipline of computer science since the 1950s, shortly after the advent of computers.

Early attempts used rule-based pattern matching, or symbolic approaches that eventually gave way to annotated, data-centric models and statistical analysis seen in machine learning and more advanced deep learning models used today. With the power of neural-networks, natural language processing achieves near human performance.

This is the kind of technology that can only be felt and not seen. Complex neural networks are difficult to reason about, much less perceive. However, it certainly leads to some interesting and fascinating outcomes. And, given modern developments that led to NLP's current state of maturity, it can certainly play a useful role when combined with assistive technologies, like language translation.

The messaging app uses a very simple interface for this function.

```
public interface LanguageTranslator {
  String translate(String text, IsoLanguage inLanguage, IsoLanguage toLanguage);
```

```
}
```

Given a body of text in a given language (`inLanguage`) , we simply want to convert the text to the specified language (`toLanguage`). The function then returns the translated text in the specified language.

Not surprisingly, the key to getting the best results is in the prompt:

"*Translate the written text {text} in {inLanguage} to {toLanguage}. Return only the translated text enclosed in square brackets: [...]. Do not provide an explanation or pronunciation. If the text does not need to be translated then simply return the text.*"

This prompt went through several iterations and tuning before I settled on the above. I am certain it could benefit from further refinements.

Regardless of my many attempts, AI models couldn't quite grasp that their only job was to translate the text, and nothing more. But, if the "text" input was not precise, used slang, swear words, or as I mentioned in the beginning of this chapter, a particular language did not have a direct translation for a word, AI felt compelled to explain. These explanations would end up in the chat messages exchanged between users, so I added "guard rails", like using square brackets to enclose the translated text, which is easy to parse unless users uncommonly use brackets in their messages. The implementation of the AI-based `LanguageTranslator` interface used by the messaging app appears as follows:

```
public String translate(String text,
    IsoLanguage inLanguage, IsoLanguage toLanguage) {
 if (isTranslatable(text, inLanguage, toLanguage)) {
  if (inLanguage.isNotEqualTo(toLanguage)) {
   TranslationKey translationKey =
    TranslationKey.from(text, toLanguage);
   return cache.computeIfAbsent(translationKey, key -> {
    // perform translation
   });
  }
 }
 return text;
}
```

The i `sTranslatable(..)` method simply checks that the text to translate is not empty, and `inLanguage` and `toLanguage` are not null. Next,

`inLanguage.isNotEqualTo(toLanguage)` is a simple optimization to make sure we are not asking the AI to do unnecessary work. As a result, the AI never receives text that does not require a translation if `translate(..)` is mistakenly called with the same form and to languages. Additionally, I cached the translations that were performed using the text and `toLanguage` as key. This optimization prevents the same text from being translated to the same language twice, reducing the cost of AI model interactions. Less is more.

Finally comes the part where Spring AI is used to interact with the configured AI model, which again, was using Meta's Llama3.2 Ollama model to translate the text:

```
PromptTemplate promptTemplate =
    new PromptTemplate(MESSAGE_TRANSLATION_PROMPT_TEMPLATE);
promptTemplate.add("text", text);
promptTemplate.add("inLanguage", inLanguage);
promptTemplate.add("toLanguage", toLanguage);
// Translate ChatMessage using AI
String translatedMessage = getChatClient().prompt()
    .messages(promptTemplate.createMessage())
    .call()
    .content();
TextMessage translatedTextMessage =
    TextMessage.from(translatedMessage);
TextMessage noMonologueTranslatedTextMessage =
  getMonologueRemover().removeMonologue(translatedTextMessage);
return noMonologueTranslatedTextMessage.getText();
```

In the code above, you can see that it leveraged Spring AI's `PromptTemplate` API to inject the text to translate along with the given and target languages of the translation. From there, it is a simple matter to use Spring AI's `ChatClient` API to prompt the AI model and receive the translated text in the generated response.

I created a separate component with the `MonologueRemover` strategy interface that was responsible for removing the AI model's persistent "explanations". This particular implementation uses square brackets, but since it is a strategy interface, you can use any implementation that best suits your use case.

The other types, such as `TranslationKey`, the `TextMessage` domain model types and `IsoLanguage` were use to develop a simple, clean interface and implementation of the `LanguageTranslator`.

It's worth noting that it helped to give the AI more "context" by declaring the language preferred by the user as the language in which the given text was written. Although, the app does not validate that the text was actually written in the preferred language. This could possibly be another use case of AI: identify the language of the given text as written by the user. This is a function used by Google Translate, for instance.

Developing a robust messaging application, like chat, is not as trivial as it might first seem. I thought of many such enhancements during development. But, if I built everything I thought about, I'd still be developing the app, and you wouldn't be reading this book. You could write an entire book on developing messaging apps by itself.

Here is the result of language translation rendered in the app:

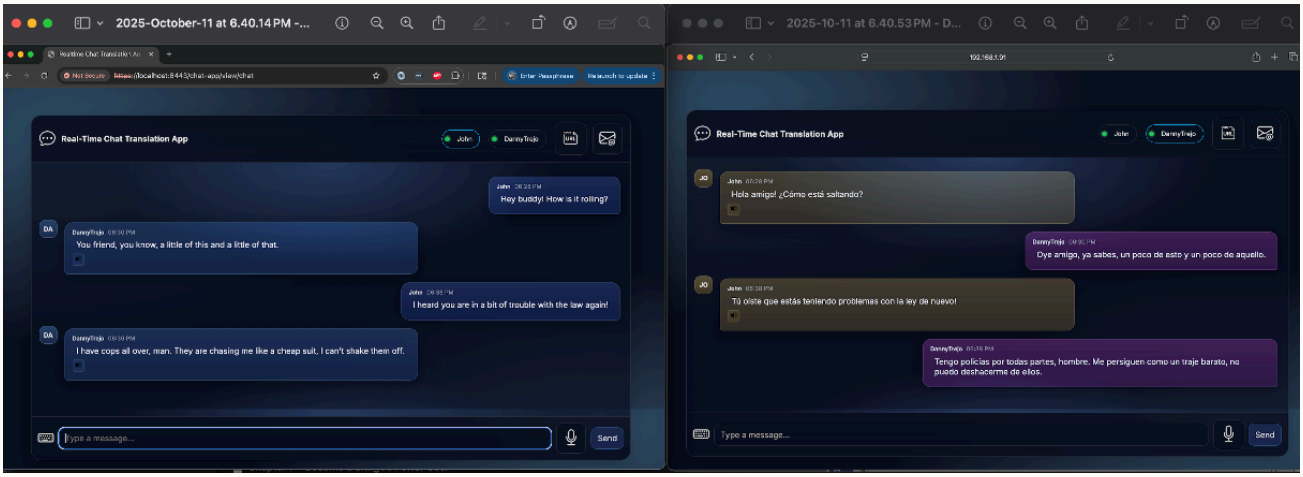


Figure 4.2: Chat Session

It's always nice when I can catch up with mis amigo, Danny Trejo. Nice of him to drop by.

The app exhibited decent performance with no noticeable latency in most cases. Network would certainly be a significant factor, but the app is using Ollama locally, by default. Also the quality of language translation will vary by AI model. I recommend that specialized AI models trained for such uses cases be used for the highest quality.

Now, let's look at the next AI use case applied by the messaging application, Audio transcription, also known as speech-to-text.

Audio transcription (Speech-to-Text)

While chatting, users have the option to verbally speak their message instead.

The interface presents users with a mic button used to record a voice message rather than always requiring users to type their message.

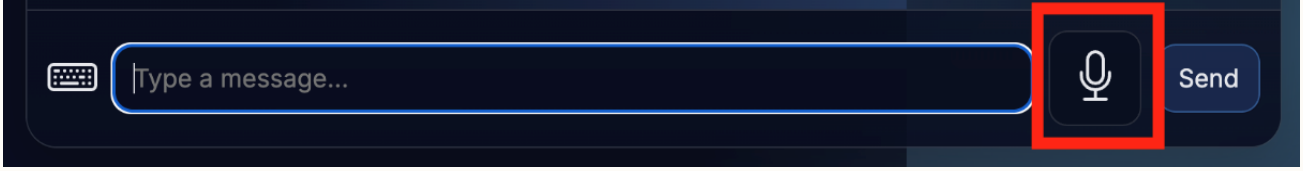


Figure 4.3: Record Audio Message

Allowing users to record a voice message rather than type one requires access to their system mic. Due to Web browser privacy controls, access to system resources, such as a mic or camera, requires the users' consent. Users will be prompted by their Web browser for their approval when they click on the mic button, which invokes JavaScript to record audio using the system's mic.

```
const mediaStreamPromise = navigator.mediaDevices.getUserMedia({ audio: true });
```

Additionally, Apple Safari, Google Chrome, Firebox and other Web browsers require the Web application to be served from a secure, encrypted connection using HTTPS in order to access system resources, such as a mic or video camera. Without a secure connection, users are not even prompted for their consent. As a result users will be unable to record voice messages using their mic.

> TIP
>
> For ease of use during testing, Google Chrome technically does not require the Web application to be served from a secure connection when accessed from localhost. However, it is always a best practice to secure the connection.

It's a simple matter to set up HTTPS even when using an embedded Web server, such as Apache Tomcat, with Spring Boot. Refer to Spring Boot's documentation on securing the HTTP connection at https://docs.spring.io/spring-boot/reference/features/ssl.html for more details.

In a nutshell, the messaging app is using the embedded Apache Tomcat Servlet container along with PEM files to secure the connection. To generate the necessary PEM files, you would use the OpenSSL library and toolset to create a private key. From the private key, you can then create a self-signed certificate that is used by the client to secure the connection to the server.

The openssl command used to generate a private key appears as follows:

```
openssl genrsa -out server.key 2048
```

This command generates a 2048 bit RSA private key then saves the key to the file, server.key. Next, you must create a certificate request using the private key you just generated:

```
openssl req -new -key server.key -out server.csr
```

When creating the certificate request, you are prompted for additional information to identify your organization, such as country, locality, organization name, email address, etc. For production applications, you would send this certificate to a Certificate Authority (CA) to verify your organizations identity and sign your certificate. Signing your own certificate is like saying, "trust me, I am legit". No legitimate organization does that.

However, for testing purposes, we'll sign our own certificate:

```
openssl x509 -req -days 365 -in server.csr -signkey server.key -out server.pem
```

To make this even easier, I have already generated a private key and self-signed certificate located in the GitHub repository in the `src/main/resources/` directory.

Now, it is just a simple matter to configure the Spring Boot embedded Web server, such as Apache Tomcat, to use the PEM files to secure the connections. I created a Spring Boot application-`ssl.properties` file with the necessary configuration already:

```
spring.ssl.bundle.pem.chat-bundle.keystore.certificate=classpath:chat-certificate.pem
spring.ssl.bundle.pem.chat-bundle.keystore.private-key=classpath:chat-private-key.pem
server.port=8443
server.ssl.bundle=chat-bundle
```

Essentially you are defining a named SSL bundle, "*chat-bundle*", *that* uses PEM files with the server's private key and certificate. Then you set Spring Boot's `server.ssl.bundle` property to the name of the SSL bundle, "*chat-bundle*". *Now y*our embedded Apache Tomcat Web server is secure, serving requests over port 8443.

When the application is served from: `https://localhost:8443/chat-app/view/chat/join`, the chat session begins. When the user attempts to use the mic, the Web browser will first prompt the user for their permission. The user only needs to grant his or her permission to access system resources once.

On the client-side, when users activate their mic and record a voice message, we must capture the audio to send to the messaging app and transcribe it with AI. Using the JavaScript `Promise` that was returned in the `Navigator` call above, we can then create a `MediaRecorder` and register for data events:

```
mediaStreamPromise.then(mediaStream => {
 var audioData = [];

 mediaRecorder = new MediaRecorder(mediaStream);

 mediaRecorder.ondataavailable = event => {
  const eventData = event.data;

  if (eventData.size > 0) {
   audioData.push(eventData);
  }
 };
});
```

When data becomes available from the `ondataavailable` event, the audio data from the event is pushed into an array of bytes. When the user stops recording, then we must send the complete array of audio bytes to the messaging app for transcription:

```
mediaRecorder.onstop = event => {
 const audioBlob = new Blob(audioData,
  { type: "audio/webm" });
 const formData = new FormData();
 formData.append("audioMessage", audioBlob, "audioMessage.webm");
 formData.append("fileName", "audioMessage.webm");

 $.ajax({
  method: "POST",
  url: "/chat-app/api/audio/transcription",
  data: formData,
  processData: false,
  contentType: false,
  enctype: "multipart/form-data",
  success: function(audioMessage) {
   postAudioMessage(audioMessage, audioBlob);
  },
  error: function(xhr, status, error) {
   logInfo(`Failed to transcribe audio: ${error}`);
  },
```

```
  });
};
```

First, we create an `AudioBlob` from the bytes of audio captured in the array, `audioData`. The `AudioBlob` is then added to a new `FormData` object stored in a form field named, "*audioMessage*", sent in the AJAX request. The app uses jQuery's `ajax({..})` function to pass the JSON payload containing the audio data in an HTTP POST request sent to the messaging app.

After the messaging app transcribes the audio and returns a response, the `postAudioMessage(..)` JavaScript function is called to update the chat UI, posting the transcribed audio into the chat as text so users can read the message.

On the server-side, a Spring Web MVC Controller handler method receives the HTTP POST request containing the audio to transcribe:

```
@PostMapping("/api/audio/transcription")
public TextMessage transcribeAudio(
    @RequestParam("audioMessage") MultipartFile audioFile) {

  Resource audioResource = audioFile.getResource();
  AudioMessage audioMessage = AudioMessage.from(audioResource);
  TextMessage textMessage = getChatService().transcribeAudio(audioMessage);

  return textMessage;
}
```

The `transcribeAudio(..)` handler method receives a Spring Web MVC `MultipartFile` constructed from the multipart/form-data encoded HTTP POST request body. The body contains the audio blob, which is accessed from the file as a Spring Resource object. We then wrap the audio resource in a `AudioMessage` domain object and call the `ChatService` to transcribe the audio, receiving a `TextMessage` in return.

The messaging app follows the layered architecture: Controller -> Service -> Data Access Object. The `ChatService` interface is simply a Facade used to aggregate all the different service providers, such as AI provider models that perform the main functions of the messaging application, like transcribing audio.

A snippet of the interface is seen below:

```
public interface ChatService {
  AudioMessage textToSpeech(TextMessage message);
  TextMessage transcribeAudio(AudioMessage message);
  ChatMessage translateChatMessage(ChatMessage message, IsoLanguage language);
}
```

The `ChatService` contains additional operations to manage chat sessions. However, the service object primarily delegates to other service provider objects, aka *Data Access Objects* (DAO) in some contexts, to perform the main functions of the messaging app.

Using Service *Provider Interfaces (SPI) a*llows different provider implementations to be used without altering the application's code. This is very useful when trying different AI providers and models, along with employing different algorithms and strategies to interact with the different providers and models, or other services in general.

In the case of audio transcription, the `ChatService` delegates to an instance of `AudioTranscriber`. `AudioTranscriber` is a Service *Provider Interface that a*llows different implementations for transcribing audio to be used without altering the `ChatService`. Out-of-the-box, the messaging app uses a single implementation based on the `OpenAiAudioTranscriptionModel`. A different implementation might use a different AI provider model or another service altogether.

The implementation of the service method used to transcribe audio appears as follows:

```
@Service
public class AiAudioTranscriber implements AudioTranscriber {
  private OpenAiAudioTranscriptionModel model;
  @Override
  public TextMessage transcribe(AudioMessage audioMessage) {
    if (audioMessage.isNotEmpty()) {
      Resource audio = audioMessage.getResource();
      String text = model.call(audio);
      return TextMessage.from(text);
    }
    return TextMessage.empty();
  }
}
```

The snippet of code above should already be familiar to you since it is similar to the code example presented in **Chapter 3** in the section on the Audio Model API.

Using Spring AI's API, OpenAI's transcription model simply takes the audio `Resource` from the `AudioMessage` and transcribes it into text. The service wraps the generated text returned in the AI model's response in an instance of `TextMessage`, which is eventually sent back to the client. That was easy.

Now, we turn our attention to the final AI use case, the opposite of audio transcription, which is to transform text into speech, also known as TTS.

Text-to-Speech

Users have the option to listen to the audio playback of a message rather than read the message themselves.

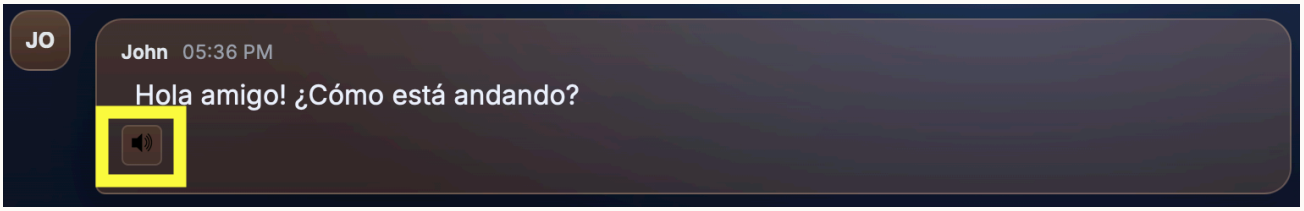


Figure 4.4: Play Audio Message

The user clicks the audio button in the chat message bubble and the app transforms the text in the chat message to audio that is available for playback.

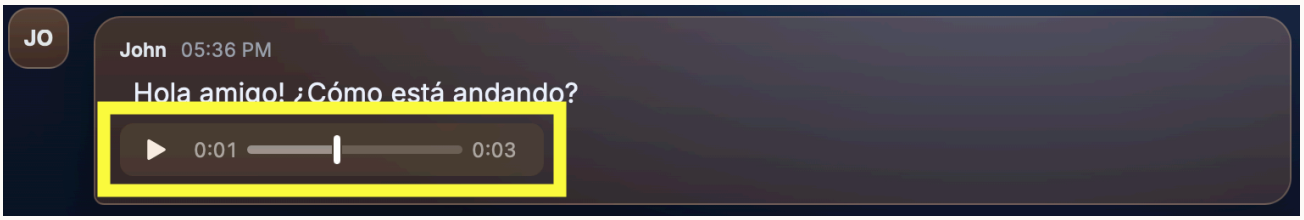


Figure 4.5: Listening to Audio Message

Clicking on the play button plays the audio message.

Sending the text message to transform from the UI to the app is pretty straightforward.

```
audioButton.innerHTML = '<img src="../img/in-progress-circles.gif" width="16" height="16">';

const textToSpeechRequest = {
  text: message.text
};

$.ajax({
  method: 'POST',
```

```
  url: `/chat-app/api/audio/generation`,
  contentType: "application/json; charset=utf-8",
  data: toJson(textToSpeechRequest),
  dataType: "json",

  success: function(response) {
   const audio = new Audio(`data:audio/mp3; base64,${response.audioData}`);
   audio.controls = true;
   $(audioMessage).empty().append(audio);
  },

  error: function(xhr, status, errorMessage) {
   const messageJson = toJson(textToSpeechRequest);

   logInfo(
    `Failed to generate audio for [${message.text}]: ${status} - ${errorMessage}`
   );
  }
 });
```

The text message is retrieved from the chat message DIV in HTML and used to construct a JSON payload that is sent as an HTTP POST request to the `/chat-app/api-audio/generation/` web service endpoint.

Once the text message is transformed to audio and returned by the app, a JavaScript `Audio` object is constructed from the base64 encoded audio data. The audio controls are then inserted into the chat message DIV to playback the message as audio.

On the server-side, the app uses the

```
@PostMapping("/api/audio/generation")
public TextToSpeechResponse textToSpeech(
   @RequestBody TextToSpeechRequest request) {
 TextMessage textMessage = request.toTextMessage();
 AudioMessage audioMessage = getChatService().textToSpeech(textMessage);
 return TextToSpeechResponse.from(audioMessage);
}
```

The Spring Web MVC `RestController` calls the `textToSpeech(textMessage)` service method from `ChatService`, which delegates to an instance of the `TextToSpeechSynthesizer` Service *Provider Interface.*

```
@FunctionalInterface
public interface TextToSpeechSynthesizer {
  AudioMessage speak(TextMessage message);
}
```

The service provider implementation is based on OpenAI TTS model, gpt-4o-mini-tts. This AI model can be called by injecting and using Spring AI's `SpeechModel` API.

The implementation is as follows:

```
public class AiTextToSpeechSynthesizer
    implements TextToSpeechSynthesizer {

  private final SpeechModel speechModel;

  @Override
  public AudioMessage speak(TextMessage textMessage) {
    SpeechMessage speechMessage =
        new SpeechMessage(textMessage.getText());

    SpeechPrompt prompt =
        new SpeechPrompt(speechMessage, newSpeechOptions());

    SpeechResponse response = speechModel.call(prompt);
    byte[] audioData = response.getResult().getOutput();

    return AudioMessage.from(audioData);
  }
}
```

We construct a `SpeechMessage` with the text message to speak. Then, the method adds the message along with `SpeechOptions` to a `SpeechPrompt` that gets sent in a request to the AI model. The AI model transforms the given text to speech.

The `SpeechOptions` are constructed with 3 configuration parameters. The options allow the application to set a voice for the generated audio, the format of the generated audio, such as MP3, and the preferred Text-to-Speech model to use when synthesizing the text as audio, as seen below:

```
private OpenAiAudioSpeechOptions newSpeechOptions() {
  return OpenAiAudioSpeechOptions.builder()
```

```
        .voice(OpenAiAudioApi.SpeechRequest.Voice.ALLOY)
        .responseFormat(
          OpenAiAudioApi.SpeechRequest.AudioResponseFormat.MP3)
        .model("gpt-4o-mini-tts")
        .build();
    }
```

From front to back, we now have a complete messaging application capable of translating language, recording audio messages, and playing audible messages back to the user.

Summary

In this chapter we looked at the implementation of a messaging app using AI to perform language translations, audio transcription and text-to-speech as users interacted with each other through the chat interface, exchanging messages in real-time.

We reviewed a few of the prompts used to generate the UI, which is quite nuanced in its rendered expression. As we learned, the key is focused prompts on isolated problems that AI models can solve readily. A few examples included generating a UI element with a certain CSS style, or a single JavaScript function to perform a particular task, like capturing audio from a mic.

We also learned how to craft a prompt containing the necessary instructions to translate text from one language to another.

Along the way we used several Spring AI features, such as the `PromptTemplate` and `ChatClient` APIs to prompt the AI model.

The `OpenAiAudioTranscriptionModel` was needed to transcribe audio, or perform speech to text transformations when users preferred to use voice messages instead of typing.

The `SpeechModel` API could be used on the receivers' end to transform a text message into an audible message that users could listen to instead of reading.

In all use cases, Spring AI enabled the application to simultaneously leverage several AI models and providers appropriate to the task at hand, and do so with elegance and precision that only Spring's programming model can.

In the next chapter, we cover another real-world use case showing one possible implementation of the popular and notorious Shazam.

Let's go "beat Shazam"!

Subscribe for a free eBook

New frameworks, evolving architectures, research drops, production breakdowns—*AI_Distilled* filters the noise into a weekly briefing for engineers and researchers working hands-on with LLMs and GenAI systems. Subscribe now and receive a free eBook, along with weekly insights that help you stay focused and informed.

Subscribe at https://packt.link/8Oz6Y or scan the QR code below.



Previous chapter
‹ Part 2: Practical Spring AI

Next chapter
Chapter 5: Beat Shazam ›